UNIVERSIDADE FEDERAL DE UBERLÂNDIA
Faculdade de Computação

RELATÓRIO TÉCNICO: ATAQUE DE SQL INJECTION (SEED LABS)

Exploração de Vulnerabilidades e Contramedida em Aplicação Web

Laboratório de Segurança Web – SEED Labs (Ubuntu 20.04)

Autor: Guilherme Carvalho Santos – 12421ECP020
Professor: Kil Jin Brandini Park

Uberlândia, 16 de setembro de 2026

SUMÁRIO

1 Introdução....3

2 Ambiente Utilizado....3

3 Familiarização com o Banco de Dados....4

4 Injeção de SQL em Comando SELECT....5

4.1 Identificação do Código Vulnerável....5

4.2 Autenticação como Administrador (Tasks 2.1 e 2.2)....5

4.3 Tentativa de Encadeamento de Comandos (Task 2.3)....6

5 Injeção de SQL em Comando UPDATE....7

5.1 Alteração do Próprio Salário (Task 3.1)....7

5.2 Alteração do Salário de Terceiro (Task 3.2)....8

5.3 Alteração da Senha de Terceiro (Task 3.3)....8

6 Contramedida: Prepared Statements....9

7 Problemas Enfrentados e Soluções....10

8 Considerações Finais....11

9 Referências....11

1 INTRODUÇÃO

Este relatório documenta a execução do laboratório SQL Injection Attack Lab, disponibilizado pela plataforma educacional SEED Labs. O objetivo do trabalho é explorar, de forma prática e em ambiente controlado, vulnerabilidades de injeção de comandos SQL em uma aplicação web de gerenciamento de funcionários, compreendendo a origem da falha e, ao final, aplicando uma contramedida efetiva que a elimina.

A aplicação utilizada como alvo é propositalmente insegura: ela constrói os comandos SQL concatenando diretamente o texto fornecido pelo usuário, sem separar o código (a estrutura da consulta) dos dados (a entrada do usuário). Essa ausência de separação é a causa fundamental do ataque de SQL Injection e será explorada ao longo das tarefas descritas nas seções seguintes.

O relatório está organizado da seguinte forma: a Seção 2 descreve o ambiente utilizado; a Seção 3 apresenta a familiarização com o banco de dados; as Seções 4 e 5 demonstram, respectivamente, os ataques sobre comandos SELECT e UPDATE; a Seção 6 apresenta a contramedida baseada em prepared statements; a Seção 7 relata os problemas enfrentados e suas soluções; e a Seção 8 traz as considerações finais.

2 AMBIENTE UTILIZADO

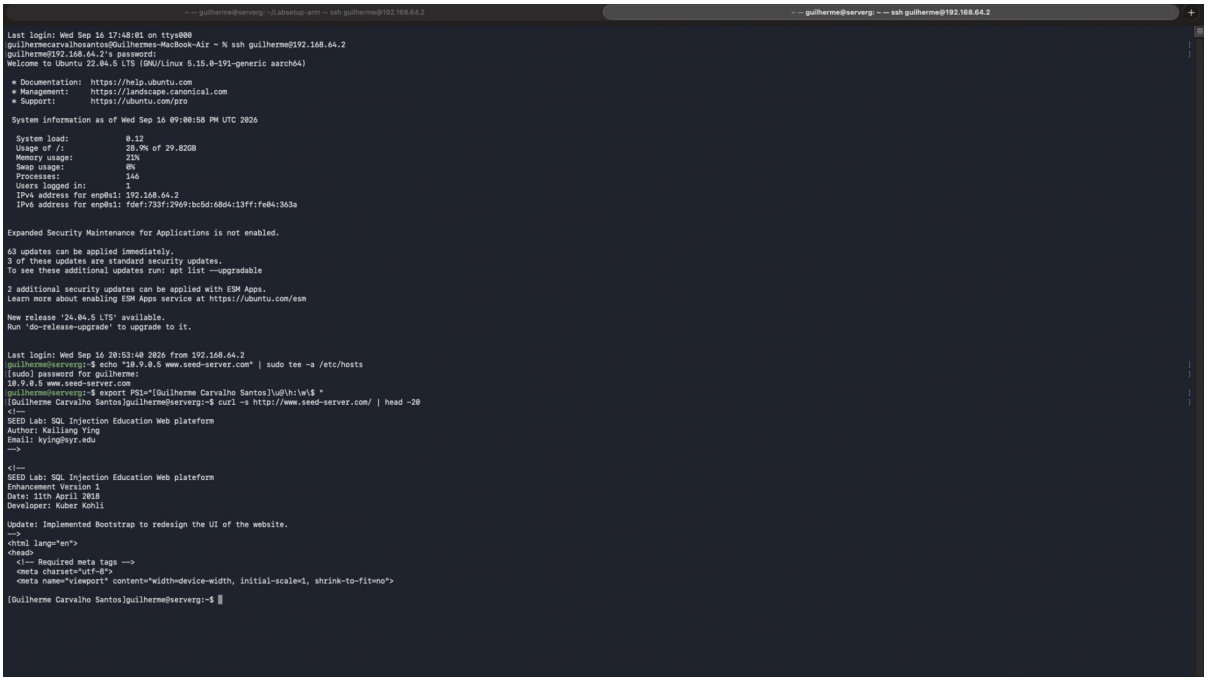
O equipamento empregado foi um computador MacBook com processador Apple Silicon (arquitetura ARM). Em virtude dessa arquitetura, a máquina virtual oficial da SEED Labs — baseada em processadores Intel/x86 e no hipervisor VirtualBox — mostrou-se incompatível. Diante disso, o ambiente foi reconstruído de forma equivalente, conforme descrito a seguir:

- Hospedeiro (host): MacBook Air com processador Apple Silicon (ARM64);
- Virtualização: UTM, alternativa ao VirtualBox compatível com Apple Silicon;
- Sistema convidado: Ubuntu Server 22.04 LTS (ARM64);
- Conteinerização: Docker e Docker Compose, executando o pacote Labsetup-arm.zip, versão ARM disponibilizada pela própria SEED Labs;
- Acesso: por meio de SSH a partir do Terminal do macOS, o que facilitou a execução dos comandos e a obtenção de capturas de tela legíveis.

A infraestrutura do laboratório é composta por dois contêineres: www-10.9.0.5, que hospeda o servidor web (Apache/PHP), e mysql-10.9.0.6, que executa o banco de dados MySQL. A aplicação responde no endereço www.seed-server.com, mapeado para o IP 10.9.0.5

por meio do arquivo /etc/hosts. A Figura 1 apresenta o ambiente em funcionamento, com a aplicação respondendo às requisições.

Figura 1 – Ambiente configurado e aplicação web respondendo às requisições



Fonte: Elaborado pelo autor (2026).

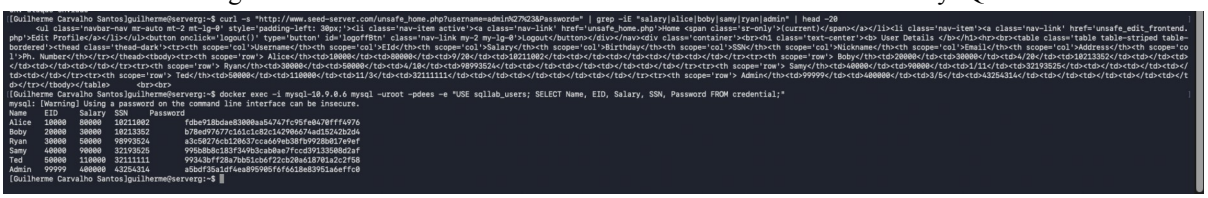
3 FAMILIARIZAÇÃO COM O BANCO DE DADOS

Antes da execução dos ataques, realizou-se o acesso direto ao banco de dados, a partir do contêiner MySQL, com o intuito de conhecer a estrutura da tabela credential, na qual são armazenados o nome, o salário, o SSN e a senha de cada funcionário. O comando utilizado foi:

```
docker exec -i mysql-10.9.0.6 mysql -uroot -pdees \
  -e "USE sqllab_users; SELECT Name, EID, Salary, SSN, Password FROM credential;"
```

A Figura 2 exibe o resultado da consulta, evidenciando os registros de todos os funcionários (Alice, Boby, Ryan, Samy, Ted e Admin) e demonstrando que as senhas são armazenadas na forma de resumo criptográfico (hash SHA1).

Figura 2 – Conteúdo da tabela credential acessado diretamente no MySQL



Fonte: Elaborado pelo autor (2026).

4 INJEÇÃO DE SQL EM COMANDO SELECT

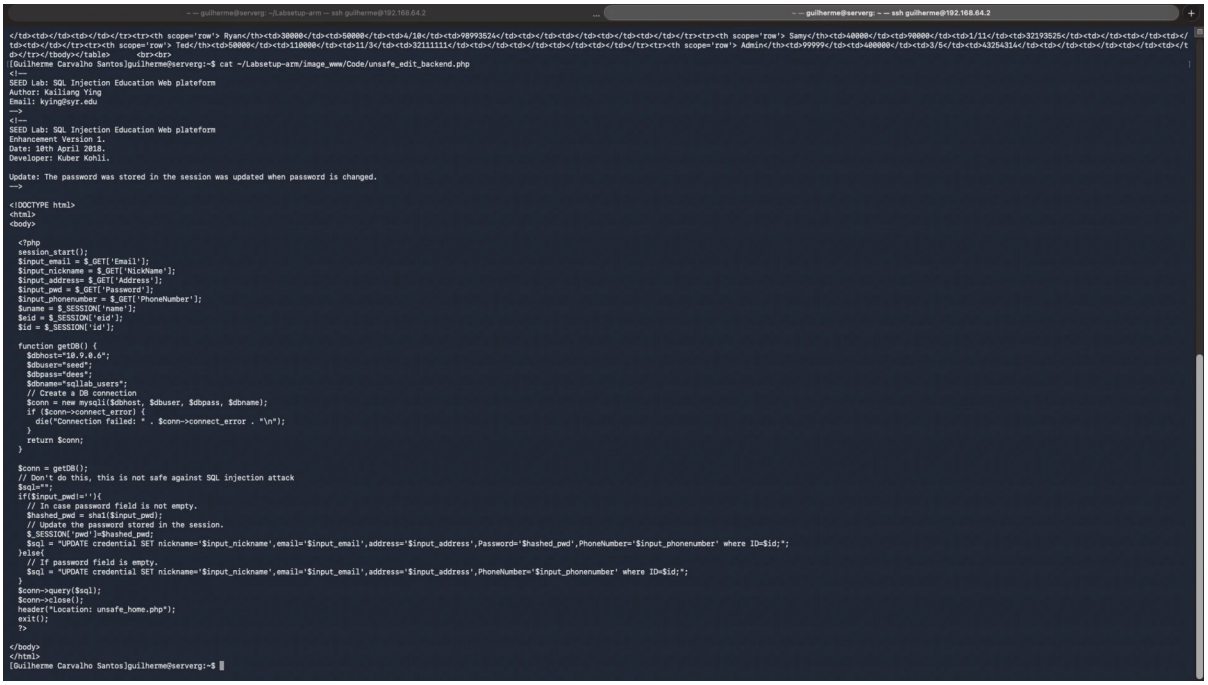
4.1 Identificação do Código Vulnerável

A página responsável pela autenticação constrói a consulta SQL concatenando diretamente o nome de usuário informado, sem qualquer tratamento, entre aspas simples, conforme o trecho a seguir:

```
$sql = "SELECT id, name, eid, salary, ... , Password
        FROM credential
        WHERE name= '$input_uname' and Password='$hashed_pwd'";
$conn->query($sql);
```

Uma vez que o valor de entrada é inserido sem sanitização, torna-se possível alterar a estrutura lógica da consulta. A Figura 3 apresenta o código-fonte vulnerável, que inclui, inclusive, um comentário do próprio desenvolvedor reconhecendo a insegurança (“this is not safe against SQL injection attack”).

Figura 3 – Trecho do código-fonte vulnerável (montagem da consulta por concatenação)



Fonte: Elaborado pelo autor (2026).

4.2 Autenticação como Administrador (Tasks 2.1 e 2.2)

Fornecendo-se, como nome de usuário, o valor admin'# , a consulta resultante passa a ser interpretada da seguinte forma:

```
SELECT * FROM credential WHERE name='admin'#' and Password='...'
```

A aspa simples (') encerra prematuramente a cadeia de texto do nome, enquanto o caractere # inicia um comentário em MySQL, fazendo com que todo o restante da linha —

inclusive a verificação de senha — seja ignorado. Assim, a consulta efetivamente executada reduz-se a:

```
SELECT * FROM credential WHERE name='admin'
```

Dessa forma, a autenticação é realizada como administrador sem o conhecimento da senha. O ataque foi reproduzido pela linha de comando (Task 2.2) por meio da ferramenta curl, na qual %27 representa a aspa simples e %23 representa o caractere #:

```
curl -s "http://www.seed-server.com/unsafe_home.php?
username=admin%27%23&Password="
```

Como resultado, a resposta retornou a página do administrador contendo os dados de todos os funcionários (nome, salário e SSN), conforme a Figura 4, confirmando o acesso indevido a informações sensíveis.

Figura 4 – Autenticação como administrador e vazamento dos dados de todos os funcionários



Fonte: Elaborado pelo autor (2026).

4.3 Tentativa de Encadeamento de Comandos (Task 2.3)

Nesta tarefa avaliou-se a possibilidade de injetar um segundo comando SQL utilizando o separador ; . A intenção foi autenticar-se como administrador e, na mesma requisição, alterar o salário do funcionário Ryan:

```
admin'; UPDATE credential SET salary='1' WHERE name='Ryan';#
```

Conforme a Figura 5, o ataque não obteve sucesso: o salário do Ryan permaneceu em 50000. A razão é uma proteção inerente à API utilizada: a função $conn->query() (mysqli::query) executa apenas um comando SQL por chamada, rejeitando comandos

adicionais posicionados após o separador ; . A execução de múltiplos comandos exigiria a função mysqli::multi_query(), não utilizada no código.

Cabe ressaltar que essa limitação impede apenas o encadeamento de comandos; ela não protege contra os demais ataques demonstrados neste relatório.

Figura 5 – Tentativa de encadeamento malsucedida: o salário do Ryan permanece inalterado



Fonte: Elaborado pelo autor (2026).

5 INJEÇÃO DE SQL EM COMANDO UPDATE

A página de edição de perfil (unsafe_edit_backend.php) também constrói o comando UPDATE por concatenação de cadeias de texto. O identificador do registro a ser alterado é obtido da sessão do usuário autenticado. Nas tarefas a seguir, assumiu-se o papel da funcionária comum Alice — autenticada por meio da mesma técnica de injeção (Alice'#) — com o objetivo de demonstrar que um usuário sem privilégios administrativos é capaz de modificar o banco de dados.

5.1 Alteração do Próprio Salário (Task 3.1)

Injetando-se, no campo NickName, o valor ',salary='80000' WHERE name='Alice';# , o comando UPDATE passa a atualizar o salário da própria Alice. A Figura 6 comprova a alteração do salário de 20000 para 80000.

Figura 6 – Alteração do próprio salário (Alice) de 20000 para 80000



Fonte: Elaborado pelo autor (2026).

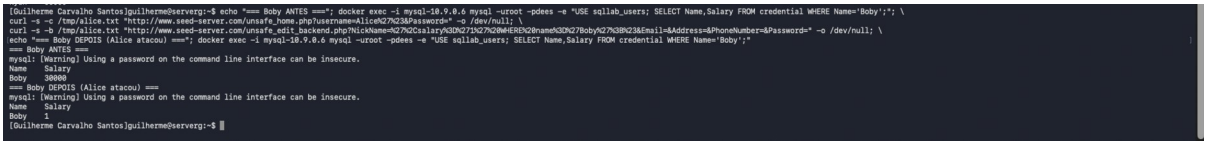
5.2 Alteração do Salário de Terceiro (Task 3.2)

Empregando a mesma técnica, porém direcionando a cláusula para outro funcionário (WHERE name='Boby'), a Alice altera um dado que não lhe pertence, reduzindo o salário do Boby para 1:

```
NickName = ',salary='1' WHERE name='Boby';#
```

A Figura 7 apresenta a comparação antes/depois, evidenciando a redução do salário do Boby de 30000 para 1 e comprovando a capacidade de adulteração de registros de terceiros.

Figura 7 – Alteração do salário de terceiro (Boby) de 30000 para 1



Fonte: Elaborado pelo autor (2026).

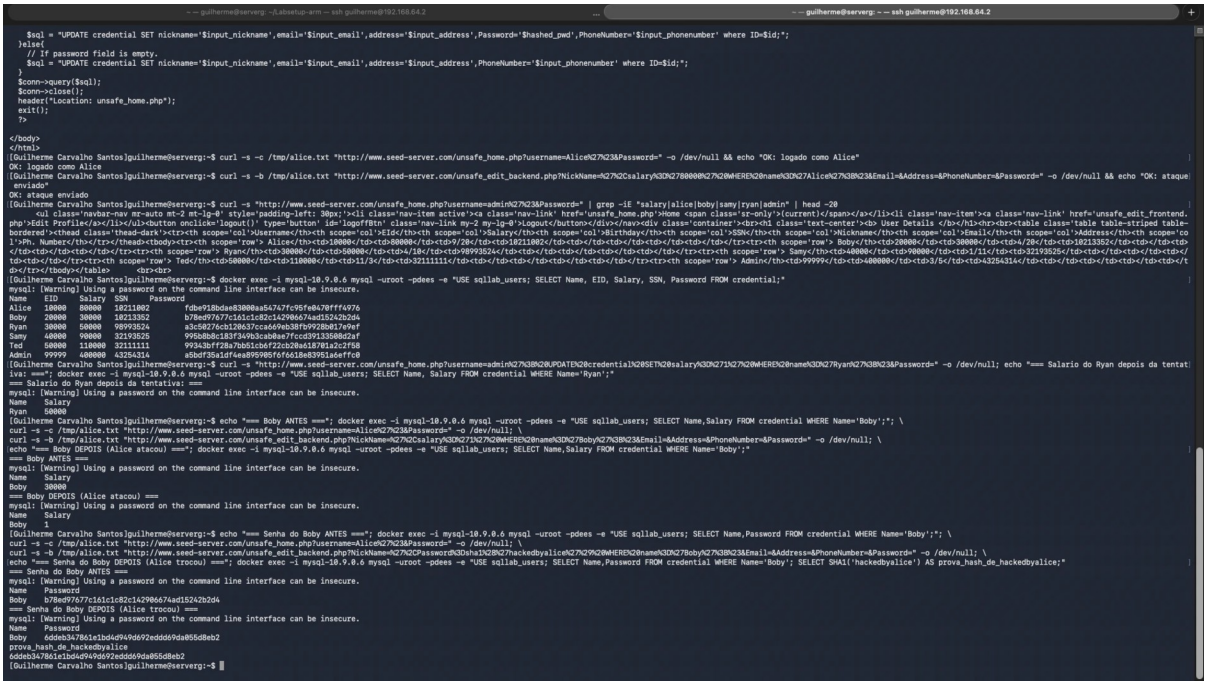
5.3 Alteração da Senha de Terceiro (Task 3.3)

O ataque de maior gravidade consiste no sequestro de conta. A Alice altera a senha do Boby, empregando a função sha1() do próprio banco de dados para gerar o hash correspondente à nova senha (hackedbyalice):

```
NickName = ',Password=sha1('hackedbyalice') WHERE name='Boby';#
```

Conforme a Figura 8, o hash da senha do Boby foi alterado e passou a coincidir exatamente com o valor de SHA1('hackedbyalice'), confirmando que a Alice definiu a nova senha do Boby e, portanto, seria capaz de acessar a conta dele.

Figura 8 – Alteração da senha de terceiro (Boby): o novo hash coincide com SHA1('hackedbyalice')



Fonte: Elaborado pelo autor (2026).

6 CONTRAMEDIDA: PREPARED STATEMENTS

A causa fundamental do SQL Injection é a mistura entre código (a estrutura do comando SQL) e dados (a entrada do usuário). A correção definitiva consiste no uso de prepared statements (comandos preparados), que enviam ao banco de dados a estrutura do comando e os dados de forma separada. Com isso, a entrada do usuário passa a ser sempre tratada como texto literal, jamais como código executável.

A consulta de autenticação foi reescrita com o uso de parâmetros (?) e da função bind_param, conforme o trecho a seguir:

```
$stmt = $conn->prepare(
  "SELECT id, name, eid, salary, birth, ssn, phoneNumber,
   address, email, nickname, Password FROM credential
   WHERE name = ? and Password = ?");
$stmt->bind_param("ss", $input_uname, $hashed_pwd);
$stmt->execute();
$result = $stmt->get_result();
```

Após a aplicação da correção no arquivo unsafe_home.php e a reconstrução do contêiner (docker compose up --build), o mesmo ataque anterior foi novamente executado. Conforme a Figura 9, o ataque passou a falhar: em vez dos dados dos funcionários, o sistema respondeu

“The account information your provide does not exist”. O texto admin'# passou a ser interpretado como um nome de usuário literal (inexistente), e não como código SQL, comprovando a eliminação da vulnerabilidade.

Figura 9 – Contramedida aplicada: o ataque admin'# não retorna mais dados



Fonte: Elaborado pelo autor (2026).

7 PROBLEMAS ENFRENTADOS E SOLUÇÕES

Durante a execução do laboratório, os principais problemas encontrados e as respectivas soluções foram:

- Incompatibilidade do VirtualBox com Apple Silicon: a máquina virtual oficial da SEED é baseada em x86 e não executa no processador ARM. Solução: utilizou-se o UTM com uma instalação própria do Ubuntu 22.04 ARM64, na qual o Docker foi instalado manualmente.
- Escolha do pacote correto: as imagens padrão do laboratório são compiladas para x86. Solução: utilizou-se o pacote Labsetup-arm.zip, versão ARM fornecida pela SEED, compatível com a arquitetura do equipamento.
- Divergência no nome do diretório: após a descompactação, a pasta chamava-se Labsetup-arm (e não Labsetup, como indicado no manual), resultando em erro de diretório inexistente. Solução: acessar o diretório com o nome correto (cd Labsetup-arm).

- Tela em branco após a primeira inicialização: a máquina virtual tentava inicializar novamente a partir da imagem de instalação (ISO). Solução: encerrar a máquina e remover (Clear) o ISO na configuração da unidade de CD/DVD do UTM.
- Ausência dos comandos dcbuild e dcup: tais comandos são apelidos (aliases) existentes apenas na máquina virtual pré-configurada da SEED. Solução: utilizaram-se os comandos equivalentes do Docker (docker compose build e docker compose up).
- Permissão do Docker: inicialmente o Docker exigia o uso de sudo. Solução: o usuário foi adicionado ao grupo docker (usermod -aG docker), seguido de nova sessão SSH.
- Resolução do domínio da aplicação: o sistema utiliza o endereço www.seed-server.com. Solução: o nome foi mapeado para o IP do contêiner (10.9.0.5) no arquivo /etc/hosts.

8 CONSIDERAÇÕES FINAIS

O laboratório demonstrou, na prática, como uma aplicação que constrói comandos SQL por concatenação de cadeias de texto fica exposta a ataques de SQL Injection. Foi possível: (i) burlar o mecanismo de autenticação e acessar o sistema como administrador sem a senha; (ii) obter indevidamente os dados sensíveis de todos os funcionários; (iii) modificar salários, inclusive de terceiros; e (iv) sequestrar contas por meio da alteração de senhas.

Evidenciou-se, ainda, que a limitação de comando único da função mysqli::query() não constitui uma defesa suficiente, uma vez que impede apenas o encadeamento de comandos. A solução adequada e definitiva é o uso de prepared statements, que separam código de dados e neutralizaram por completo os ataques, conforme comprovado na Seção 6.

Conclui-se que a validação manual de entradas é frágil e propensa a falhas. A prática segura recomendada é a utilização sistemática de consultas parametrizadas (prepared statements) em toda interação com bancos de dados, garantindo a separação entre código e dados e eliminando a superfície de ataque explorada neste trabalho.

9 REFERÊNCIAS

DU, Wenliang. SEED Labs – SQL Injection Attack Lab. Syracuse University. Disponível em: https://seedsecuritylabs.org/Labs_20.04/Web/Web_SQL_Injection/. Acesso em: 16 set. 2026.

THE PHP GROUP. mysqli::prepare – Prepared Statements. PHP Manual. Disponível em: https://www.php.net/manual/en/mysqli.prepare.php. Acesso em: 16 set. 2026.